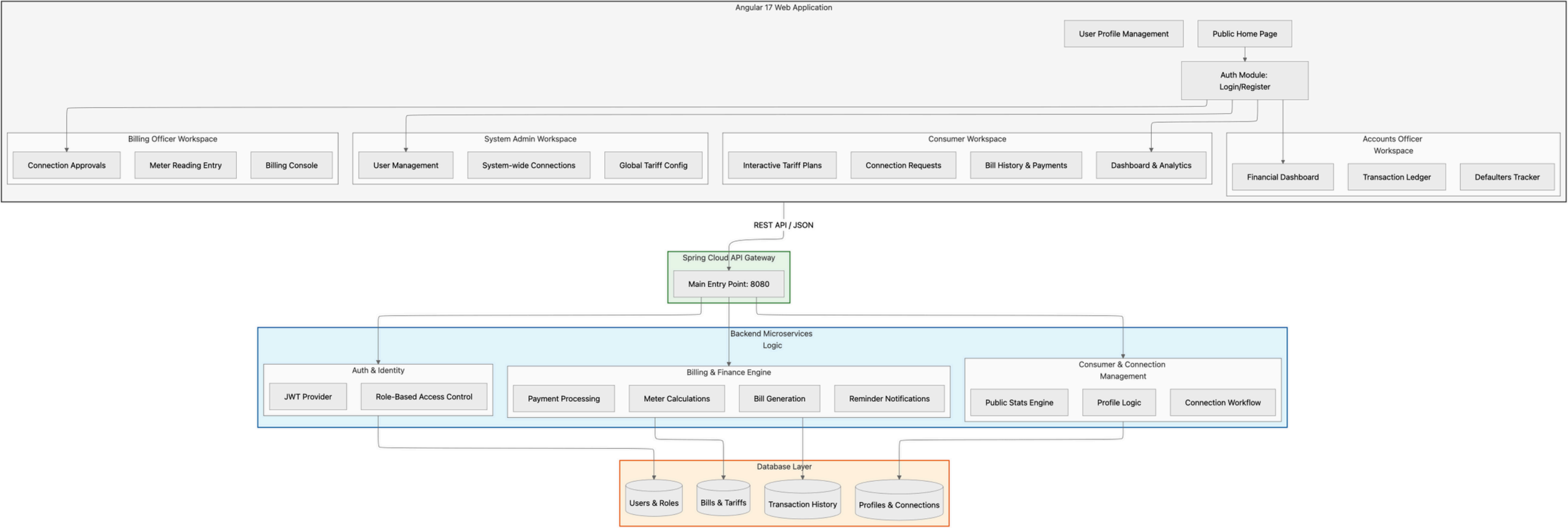


Utilix

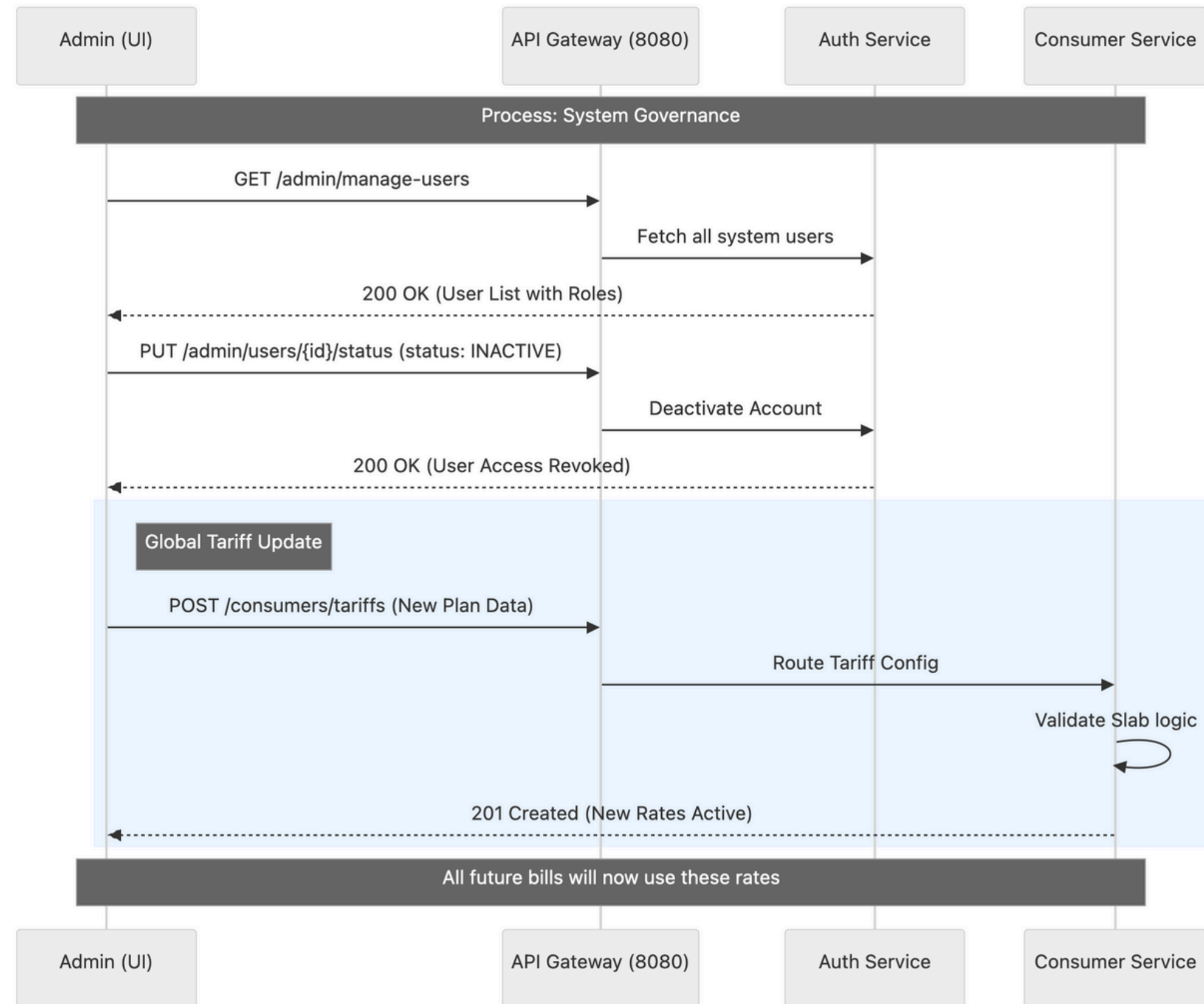
Utility Billing System

Platform for electricity, water, gas, and internet billing. Monitor usage, pay bills, and manage connections—all in one place.

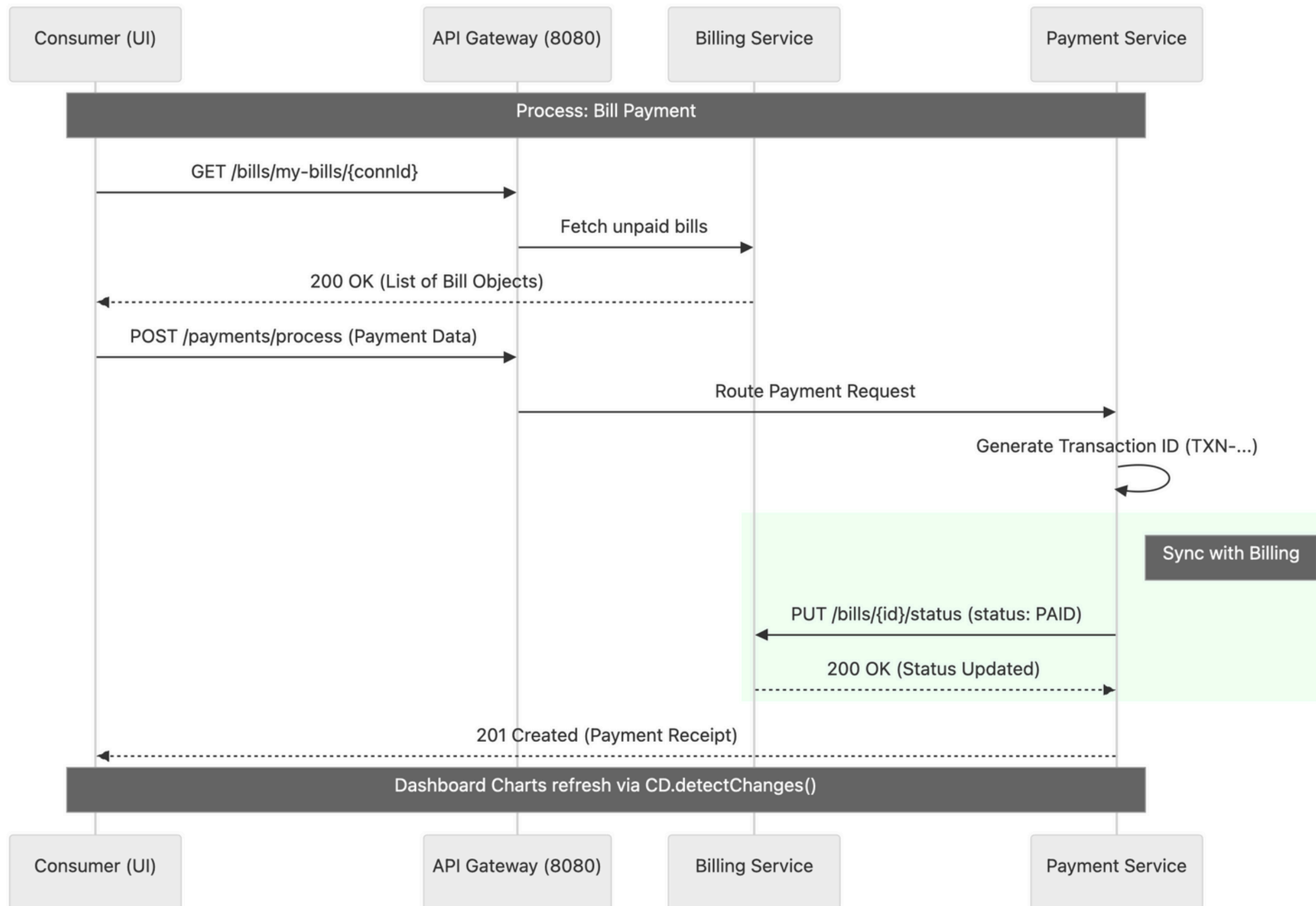
Architecture Diagram



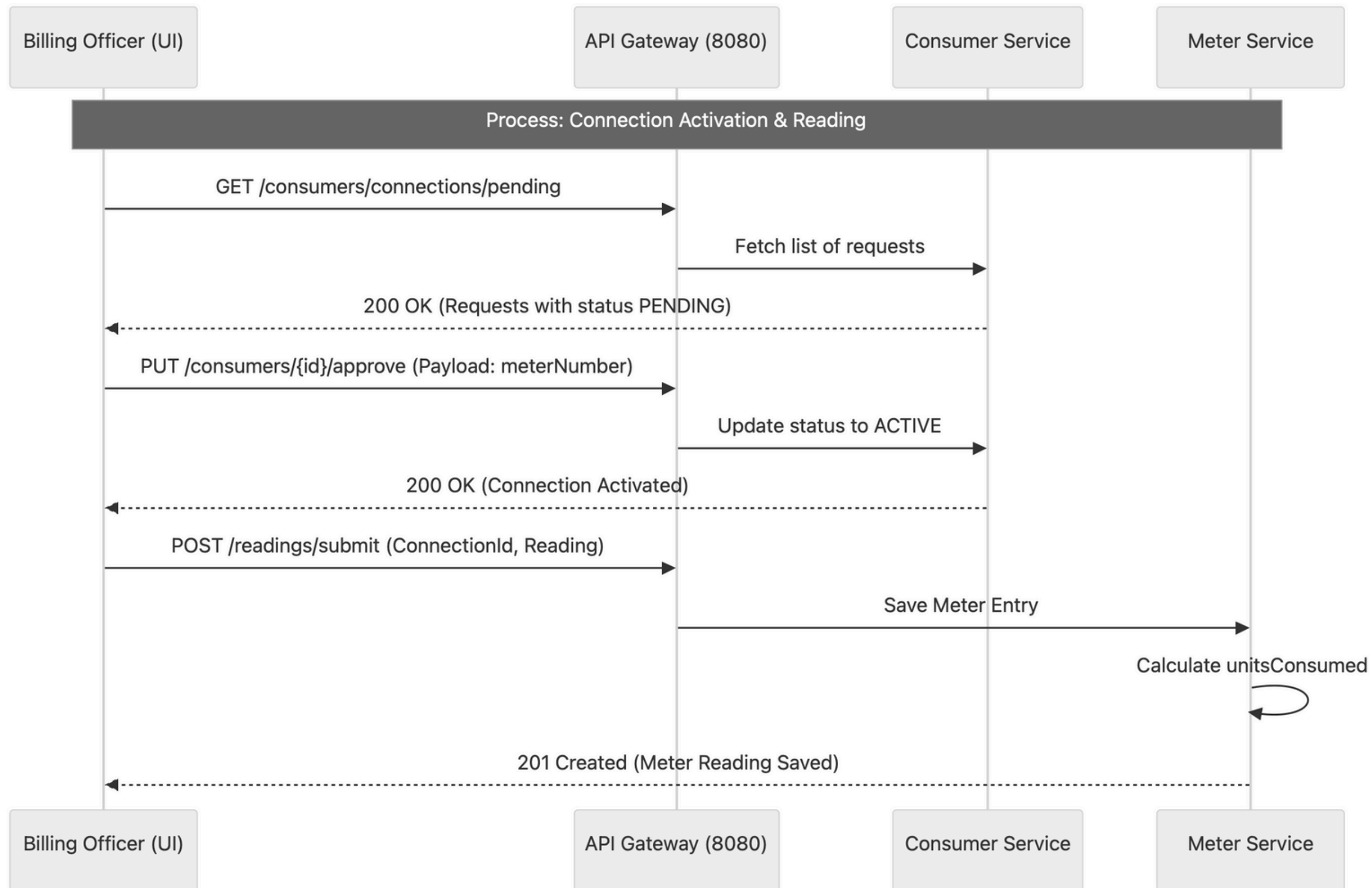
Sequence Diagram – Admin



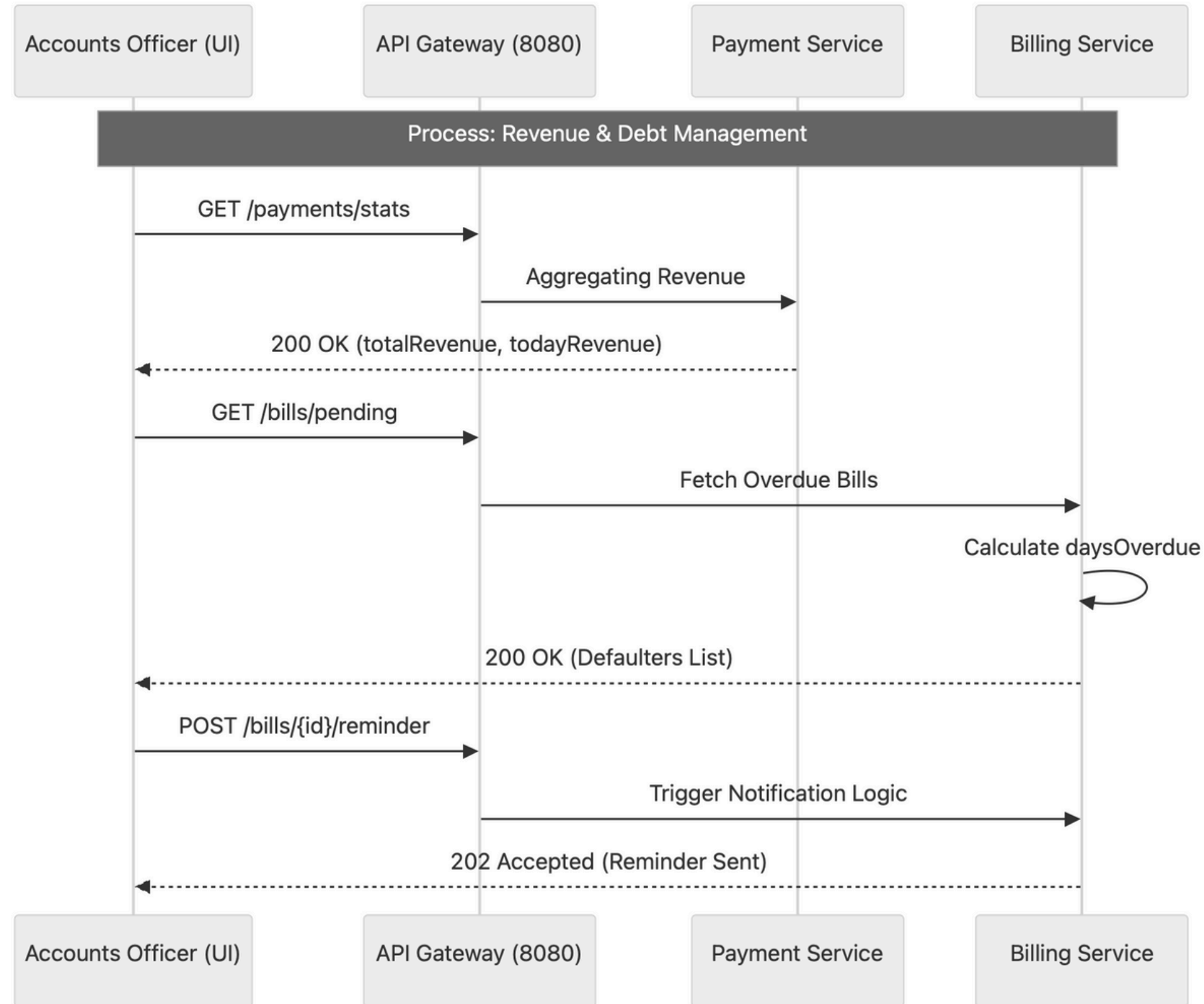
Sequence Diagram – Consumer



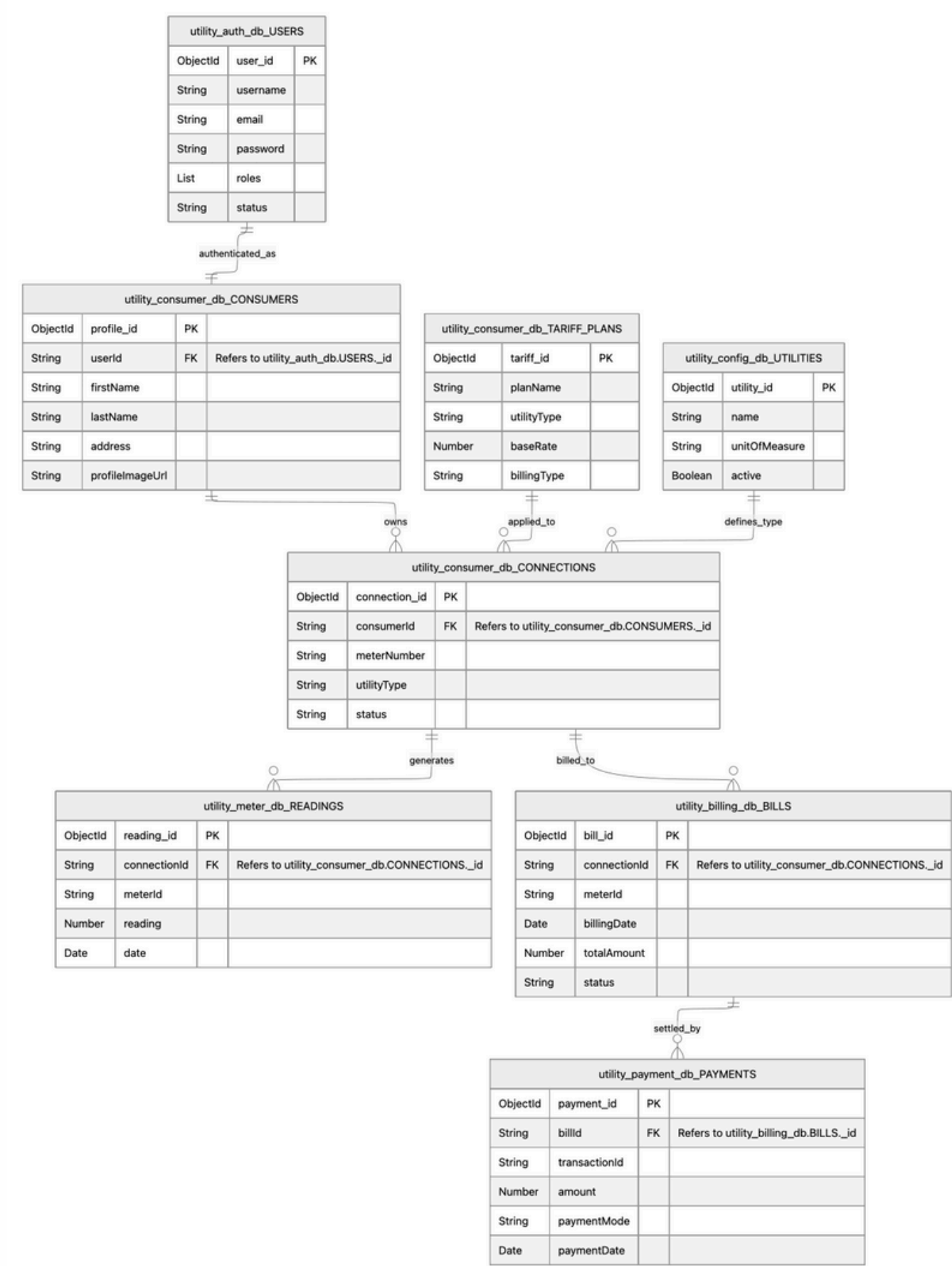
Sequence Diagram – Billing Officer



Sequence Diagram – Accounts Officer



Database Structure



Users

USERS	
ObjectId	_id
String	username
String	email
String	password
List_String	roles
Boolean	active
String	firstName
String	lastName
String	phoneNumber
String	address
String	status
String	_class

Bills

BILLS	
ObjectId	_id
String	connectionId
String	meterId
Date	billingDate
Date	dueDate
Number	previousReading
Number	currentReading
Number	unitsConsumed
Number	ratePerUnit
Number	fixedCharge
Number	taxAmount
Number	amount
Number	totalAmount
String	status
Number	fineAmount
Number	lateFeePerDay
Number	gracePeriod
String	utilityType
String	tariffPlanName
String	_class

UTILITIES

UTILITIES	
ObjectId	_id
String	name
String	unitOfMeasure
Boolean	active
String	_class

PAYMENTS

PAYMENTS	
ObjectId	_id
String	billId
Number	amount
String	paymentMode
String	status
String	transactionId
Date	paymentDate
String	_class

CONSUMER

CONSUMERS	
ObjectId	_id
String	userId
String	firstName
String	lastName
String	phoneNumber
String	address
Boolean	active
String	profileImageUrl
String	_class

owns

CONNECTIONS	
ObjectId	_id
String	consumerId
String	utilityType
String	meterNumber
String	tariffCategory
Date	connectionDate
String	status
String	_class

assigned_to

TARIFF_PLANS	
ObjectId	_id
String	utilityType
String	category
String	billingType
String	planName
String	description
Number	baseRate
String	_class

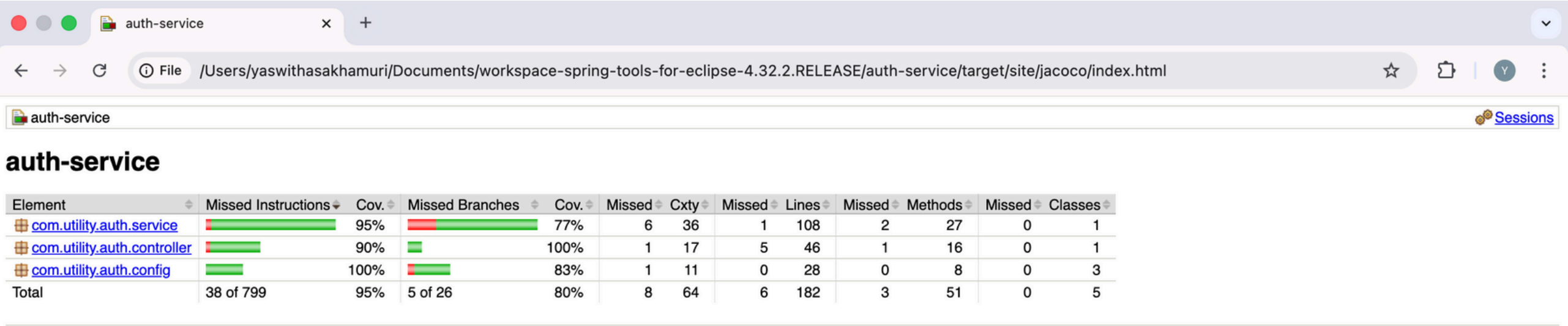
Why MongoDB? (NoSQL Choice)

- Schema Flexibility: Utility data is diverse. For example, an Internet bill has fixed subscription costs, while Electricity bills have complex slab rates (0-100, 101-200). MongoDB handles these varying document structures without requiring complex SQL joins.
- Scalability: Microservices generate a high volume of time-series data (Meter Readings). MongoDB's horizontal scaling (sharding) makes it more suitable for high-traffic utility monitoring than traditional RDBMS.
- JSON-Native: Since the entire stack (Angular to Spring Boot) communicates via JSON, using a document store eliminates the "impedance mismatch" (converting rows to objects).

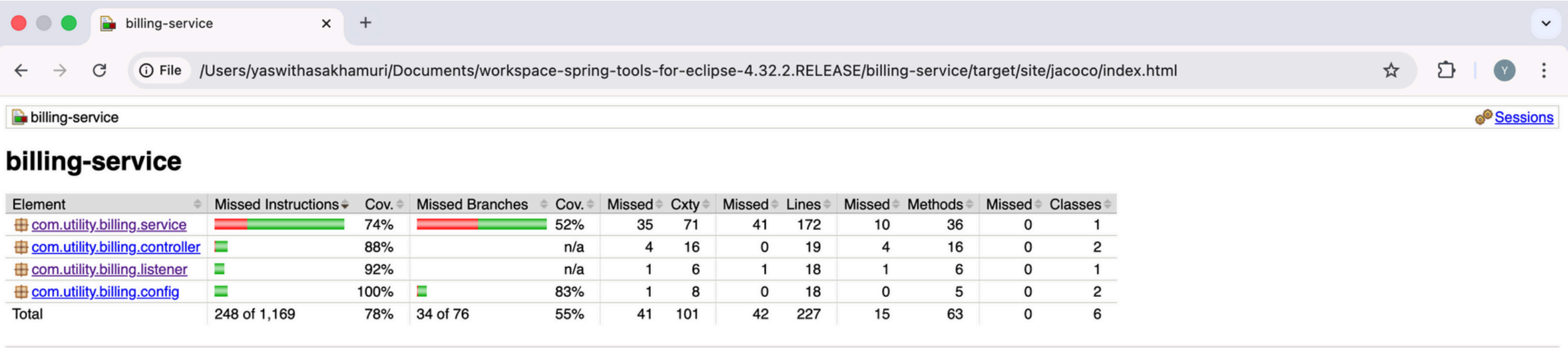
Business Decisions

- Self-Service Connection Requests: Instead of manual paperwork, users request connections via their dashboard.
Impact: Reduces administrative overhead for the utility company and speeds up customer onboarding.
- Tiered Role Hierarchy (RBAC): Separating Billing Officers (data entry) from Accounts Officers (revenue management).
Impact: Prevents fraud. A person entering meter data cannot also delete their own outstanding dues.
- Automated Transparency: Giving consumers a "Tariff Plan" tab within their dashboard.
Impact: Reduces support calls regarding "Why is my bill high?" by educating the user on slab-based rates.

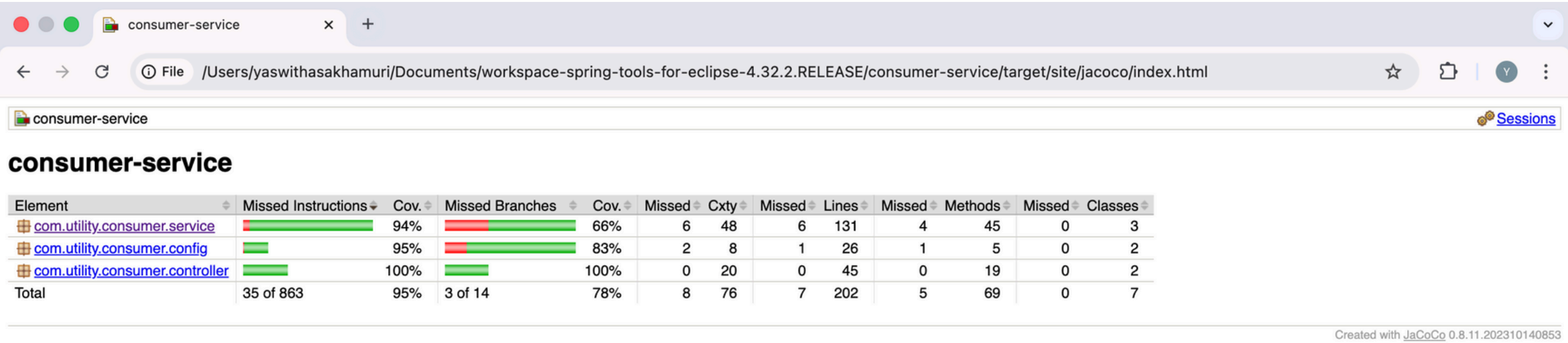
Code Coverage – Jacoco Report



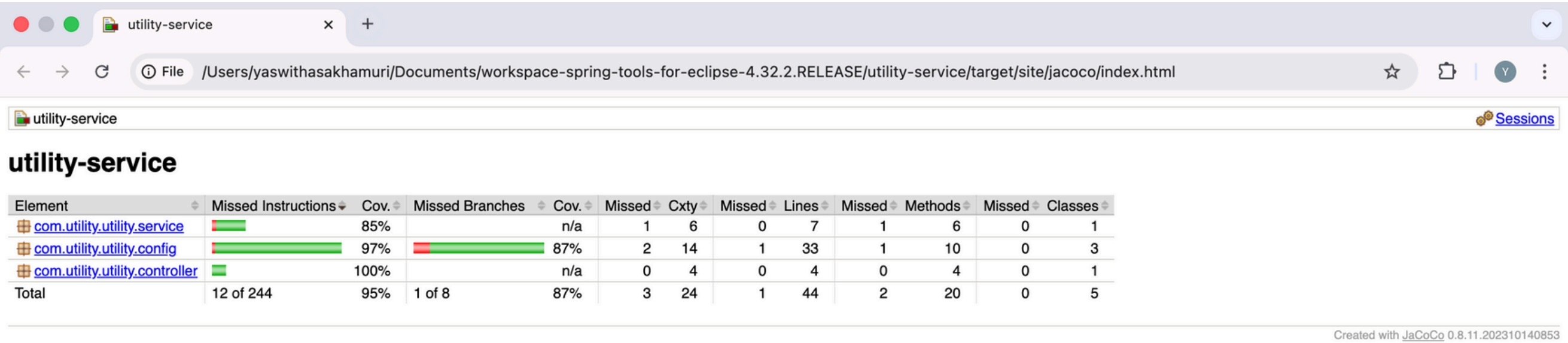
Code Coverage – Jacoco Report



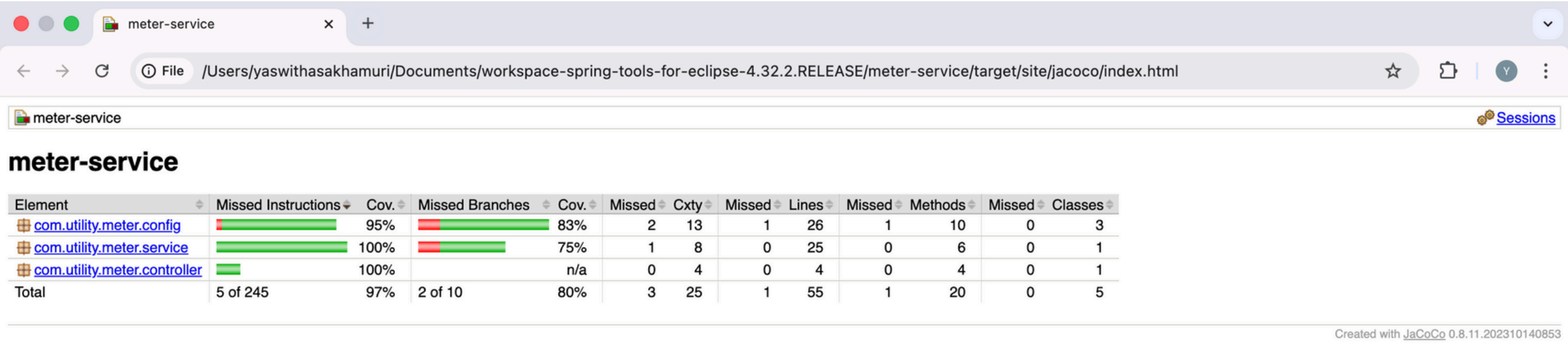
Code Coverage – Jacoco Report



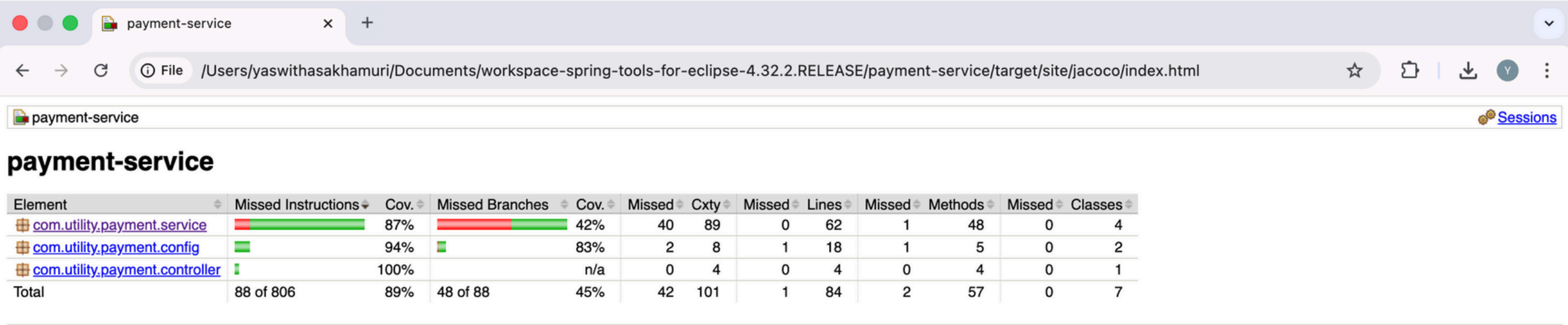
Code Coverage – Jacoco Report



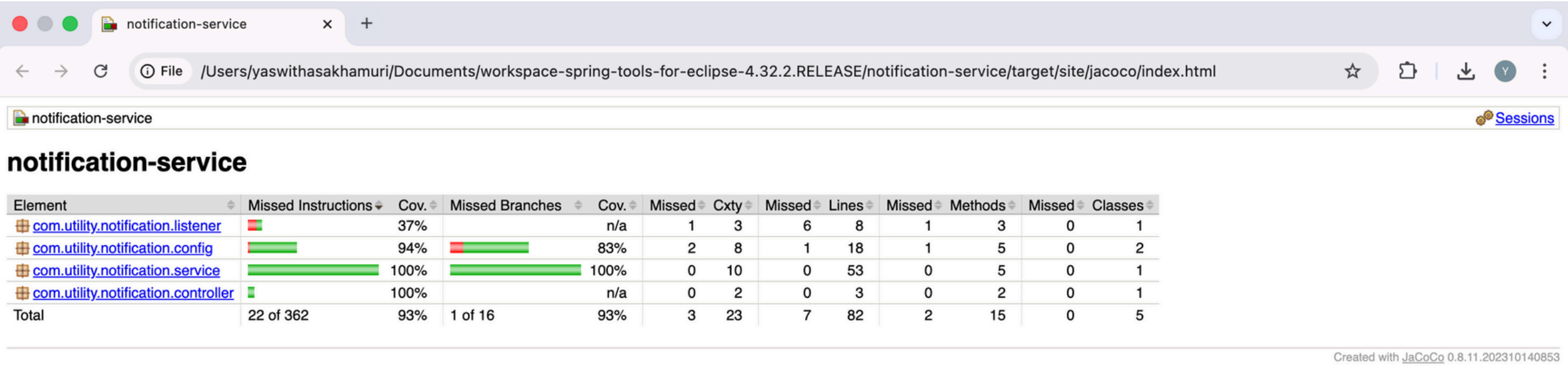
Code Coverage – Jacoco Report



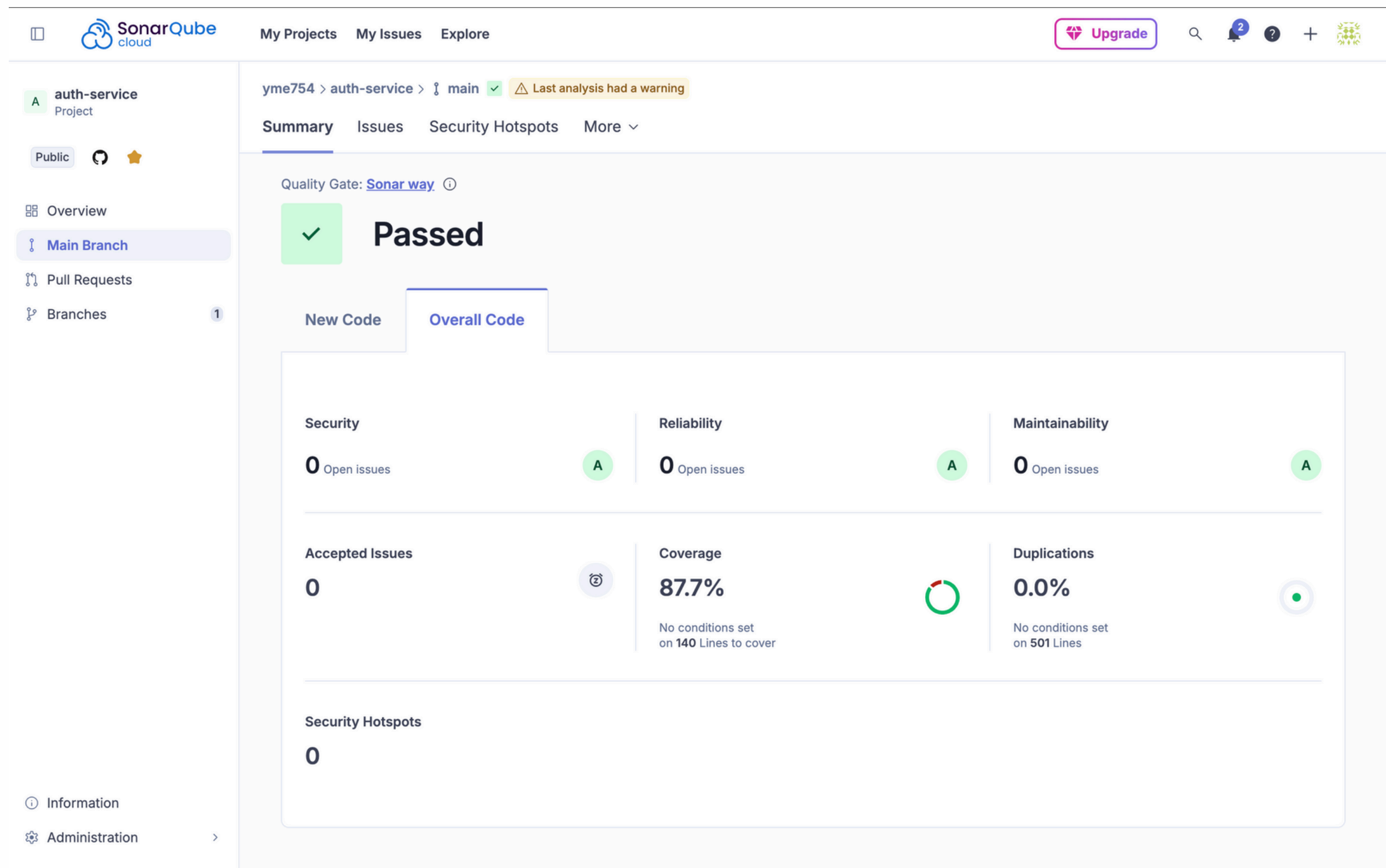
Code Coverage – Jacoco Report



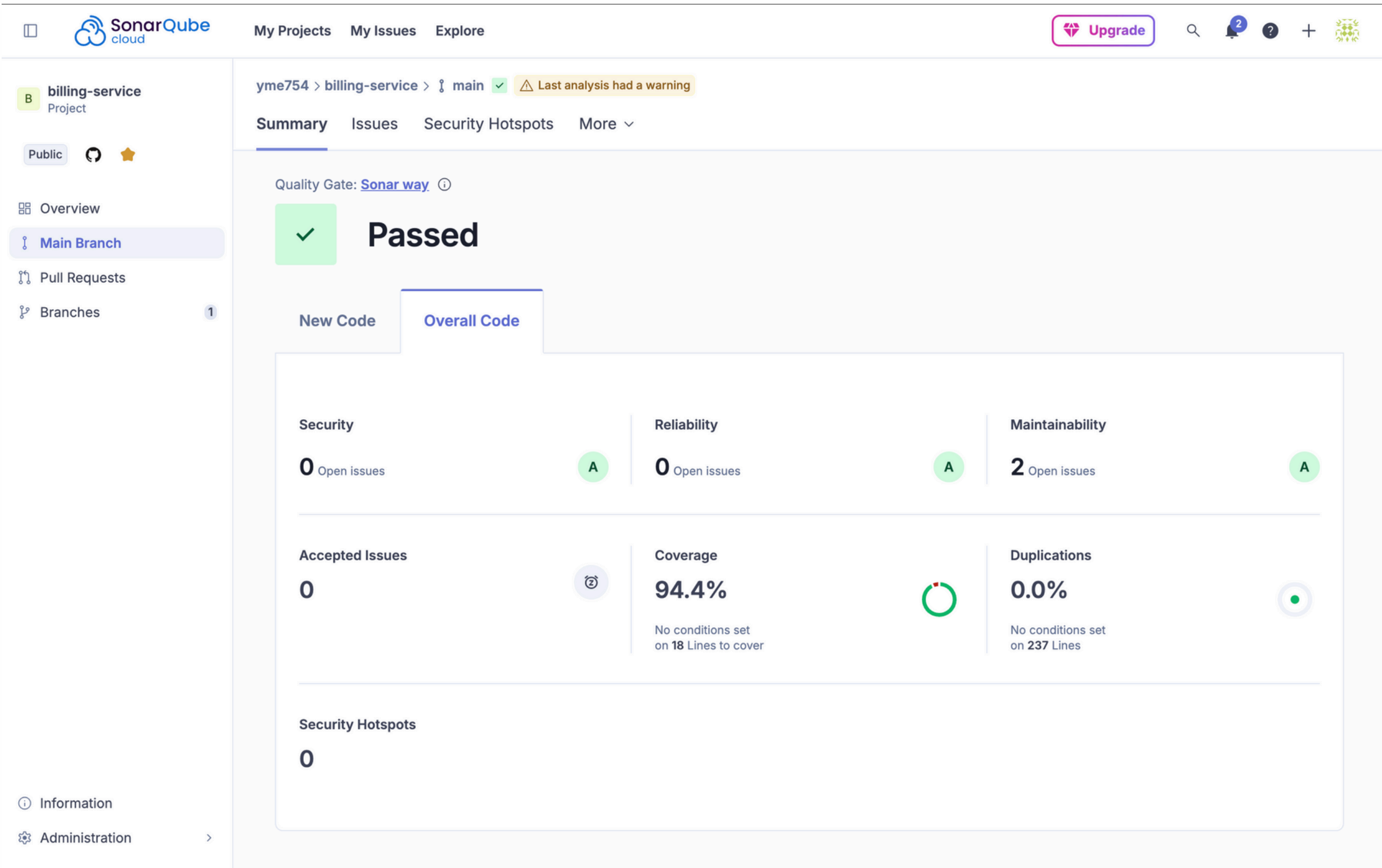
Code Coverage – Jacoco Report



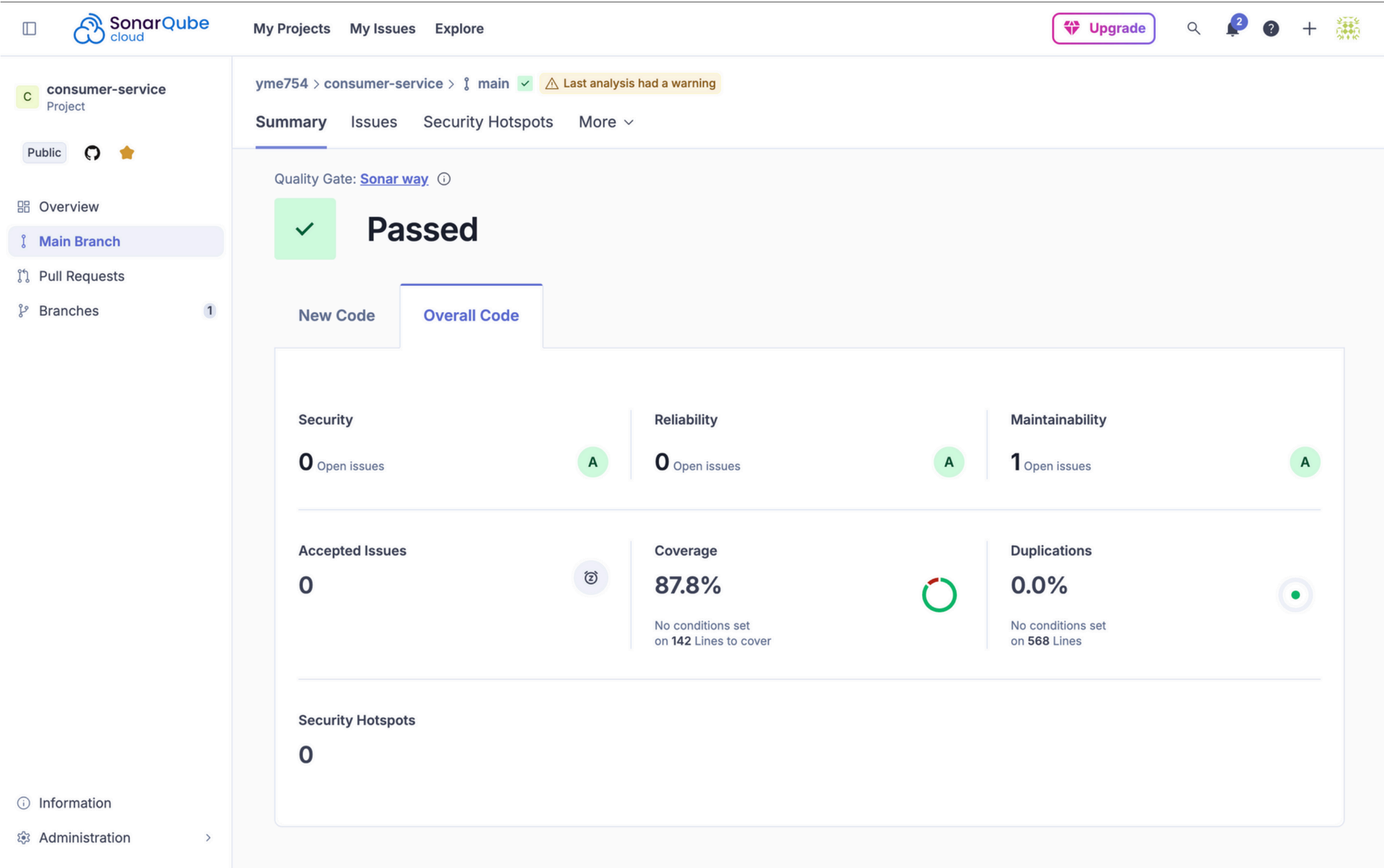
Code Coverage – SonarQube Report



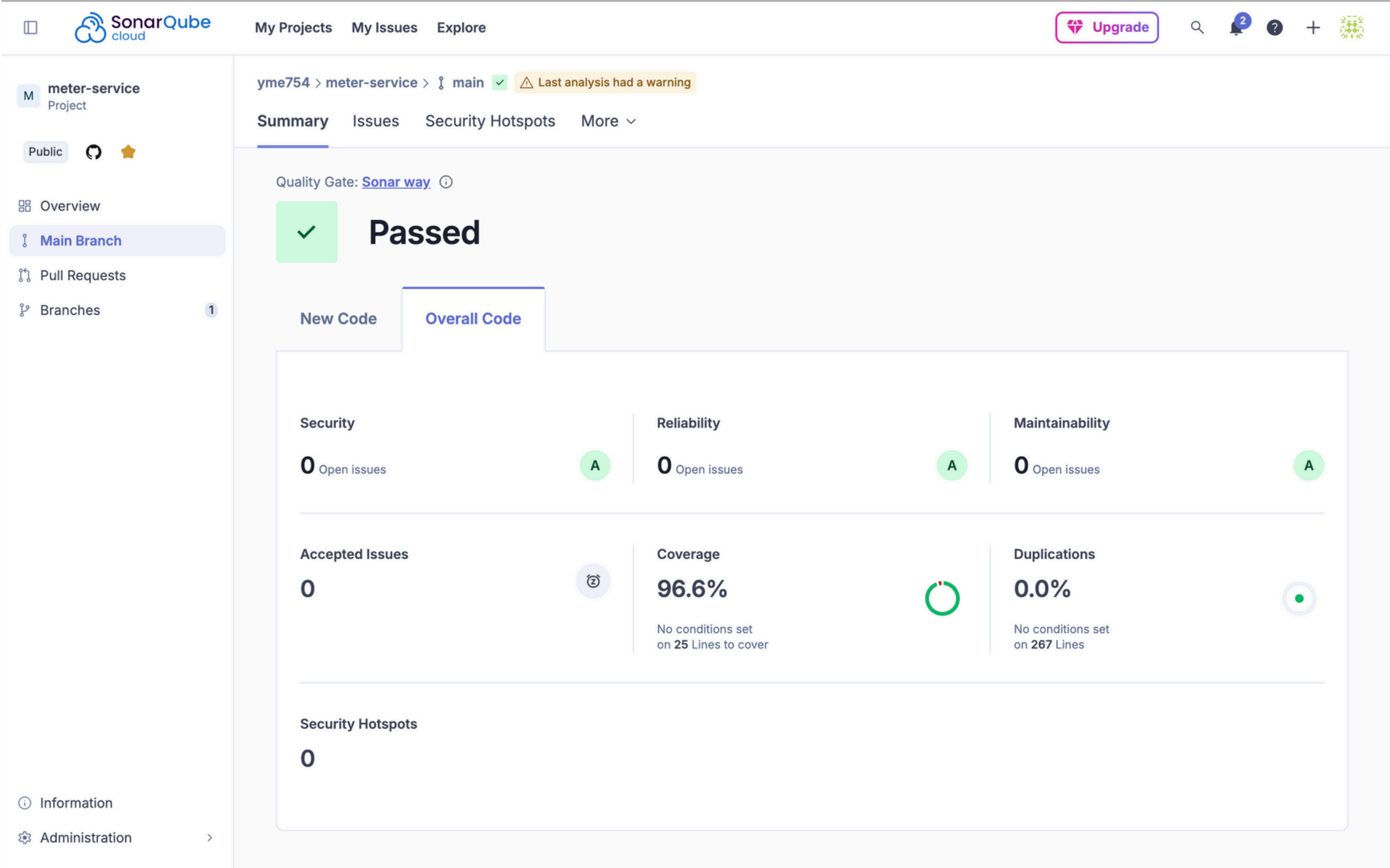
Code Coverage – SonarQube Report



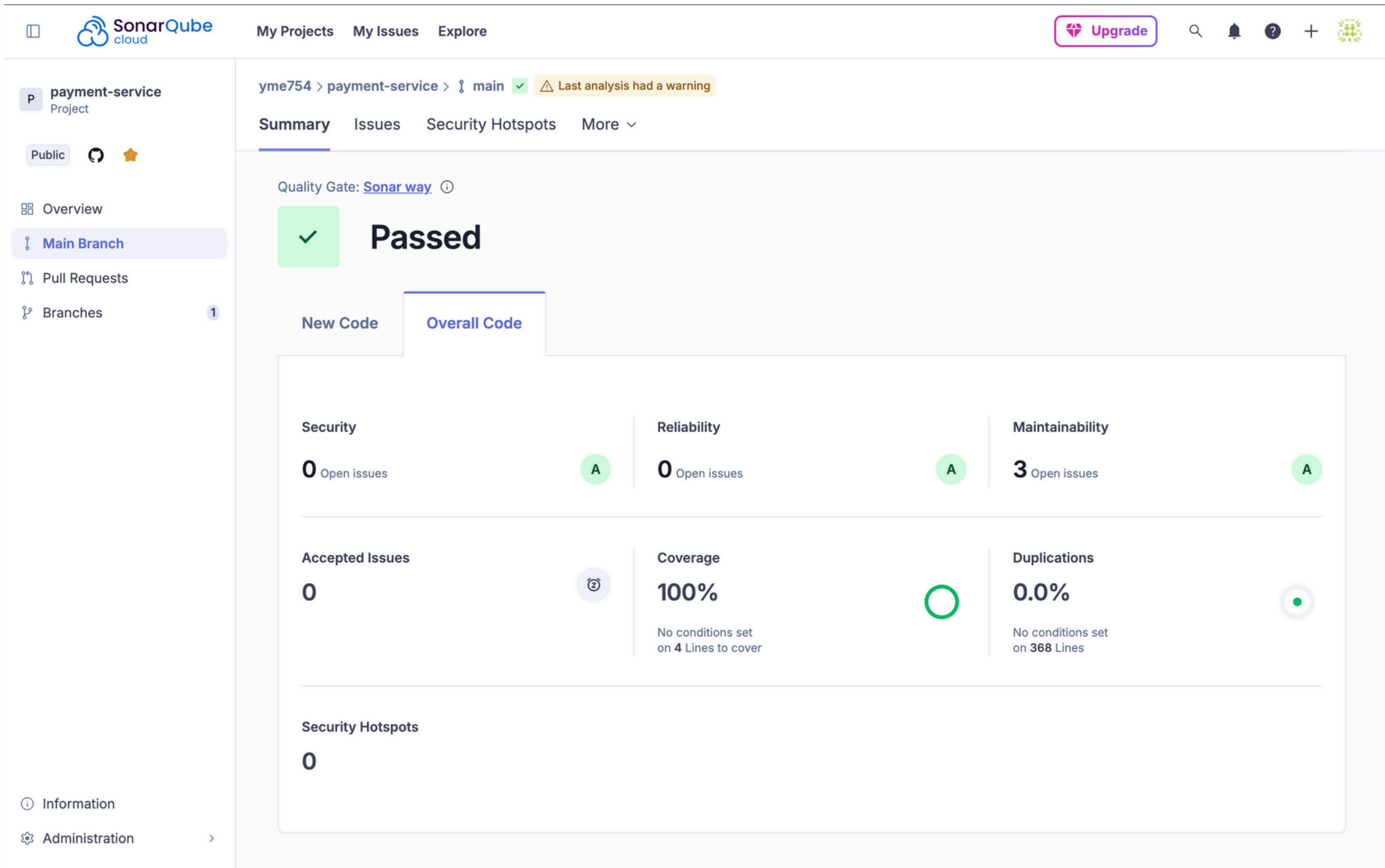
Code Coverage – SonarQube Report



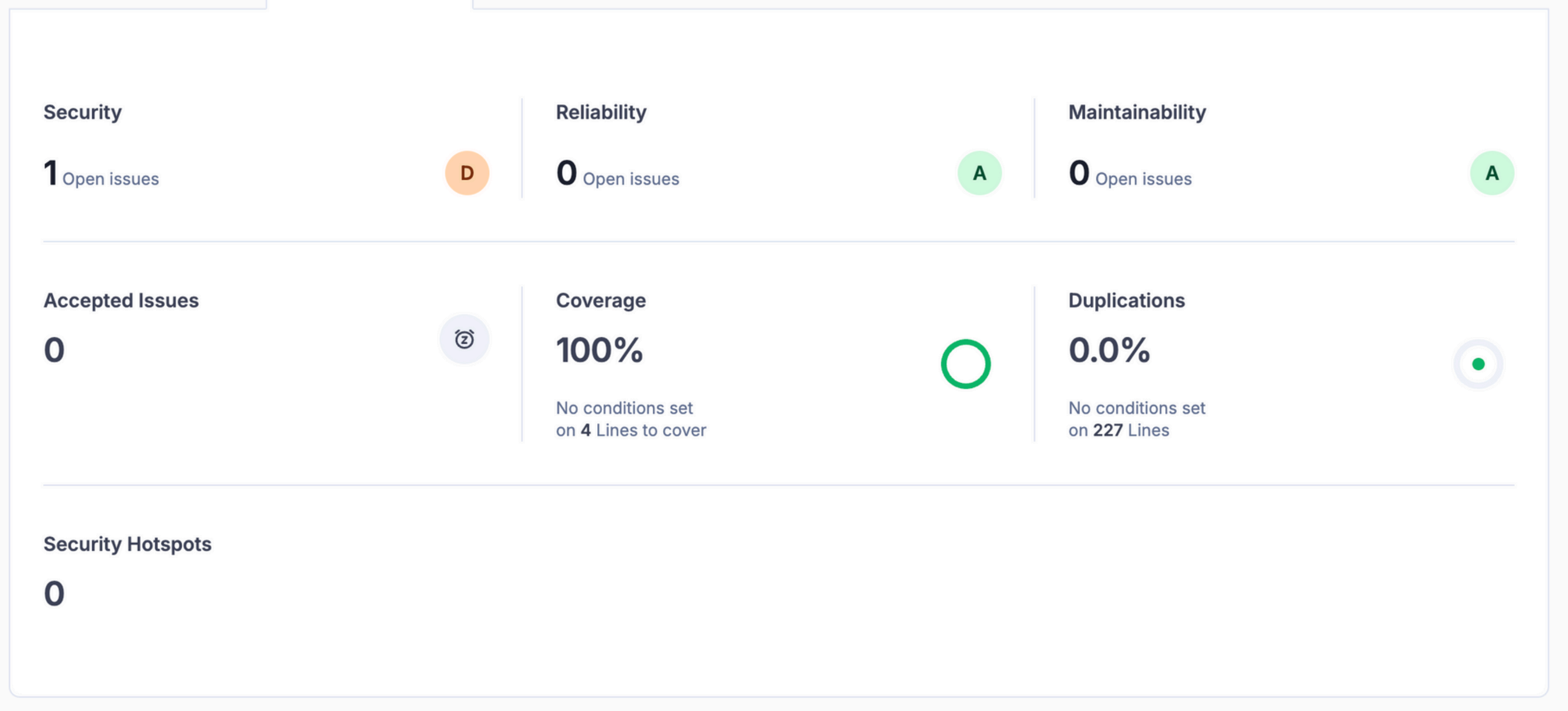
Code Coverage – SonarQube Report



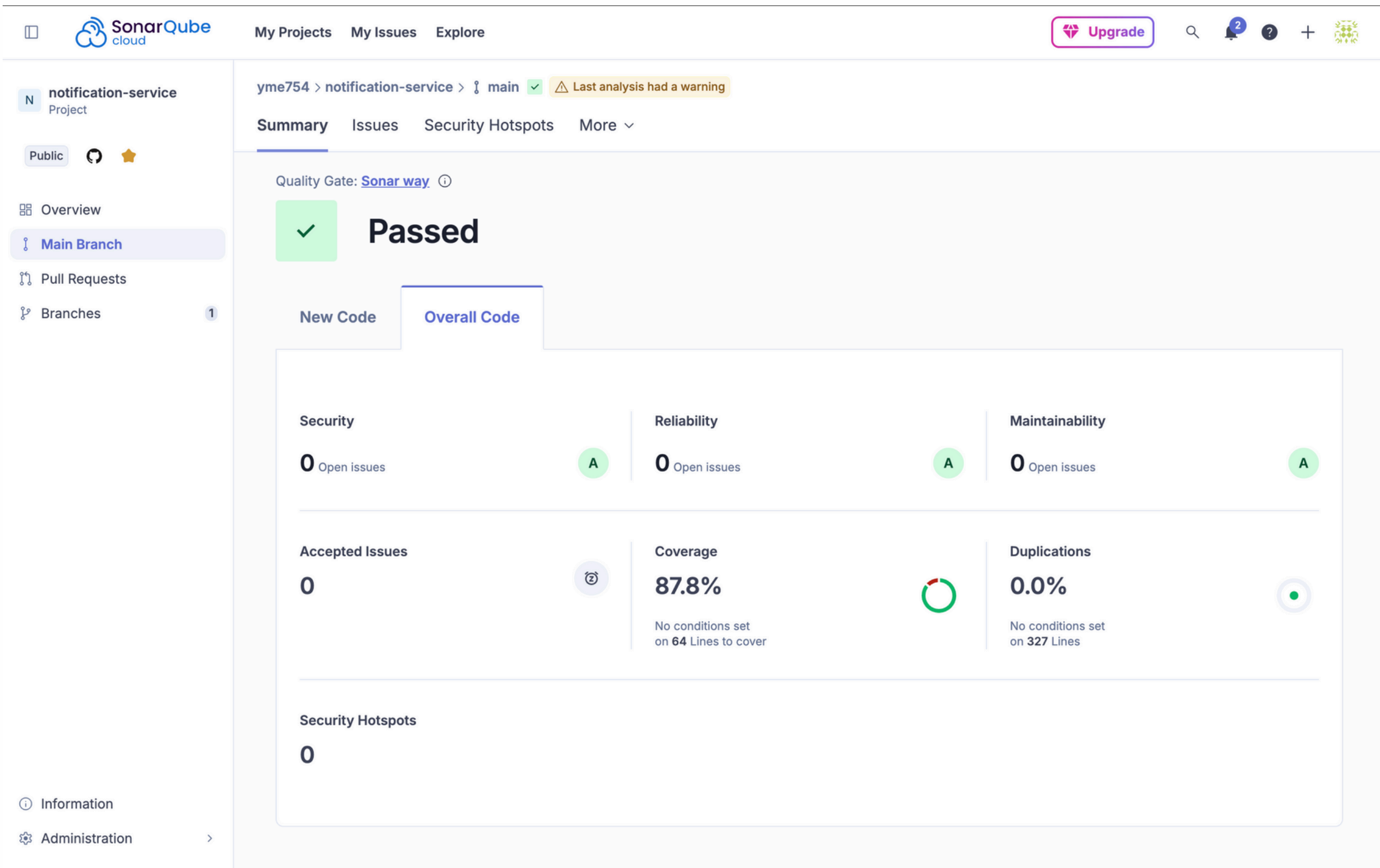
Code Coverage – SonarQube Report



Code Coverage – SonarQube Report



Code Coverage – SonarQube Report



Docker

docker desktop

PERSONAL

Search

⌘K

?

🔔

📦

⚙️

⋮

Y

Ask Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Containers

Give feedback

Container CPU usage ⓘ

54.83% / 800% (8 CPUs available)

Container memory usage ⓘ

2.93GB / 3.74GB

Show charts

Search

☰

Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Mer	Actions
☐	workspace-sprir -	-	-	-	54.76%	2.93GB / 46.42GE		⚙️ 🔴 ⋮ 🗑️
☐	mongo	9a6daf307066	mongo:7.0	27017:27017	42.91%	93.23MB / 3.83GI		⚙️ 🔴 ⋮ 🗑️
☐	discovery-ser	6cb4826dfe3b	workspace:	8761:8761	0.22%	248MB / 3.83GB		⚙️ 🔴 ⋮ 🗑️
☐	zookeeper	b9ce82fd3ceb	confluentin	2181:2181	0.14%	54.46MB / 3.83GI		⚙️ 🔴 ⋮ 🗑️
☐	kafka	33d6b3afd244	confluentin	9092:9092	4.03%	234.3MB / 512Mi		⚙️ 🔴 ⋮ 🗑️
☐	config-server	ea809b454654	workspace:	8888:8888	0.22%	162.1MB / 3.83GI		⚙️ 🔴 ⋮ 🗑️
☐	consumer-sei	7bac3200a072	workspace:	8082:8082	1.74%	326.1MB / 3.83GI		⚙️ 🔴 ⋮ 🗑️
☐	meter-service	b57e0b19ebab	workspace:	8085:8085	0.11%	249.5MB / 3.83GI		⚙️ 🔴 ⋮ 🗑️
☐	notification-s	9ae847b0a847	workspace:	8087:8087	1.7%	240.9MB / 3.83GI		⚙️ 🔴 ⋮ 🗑️
☐	utility-service	bc172e23dcfa	workspace:	8083:8083	0.16%	271MB / 3.83GB		⚙️ 🔴 ⋮ 🗑️
☐	auth-service	74b92e939e25	workspace:	8081:8081	0.23%	292.1MB / 3.83GI		⚙️ 🔴 ⋮ 🗑️
☐	payment-serv	690d06ca46c8	workspace:	8088:8088	1.76%	257.3MB / 3.83GI		⚙️ 🔴 ⋮ 🗑️
☐	api-gateway	58f538419caa	workspace:	8080:8080	0.11%	264.6MB / 3.83GI		⚙️ 🔴 ⋮ 🗑️

Showing 14 items

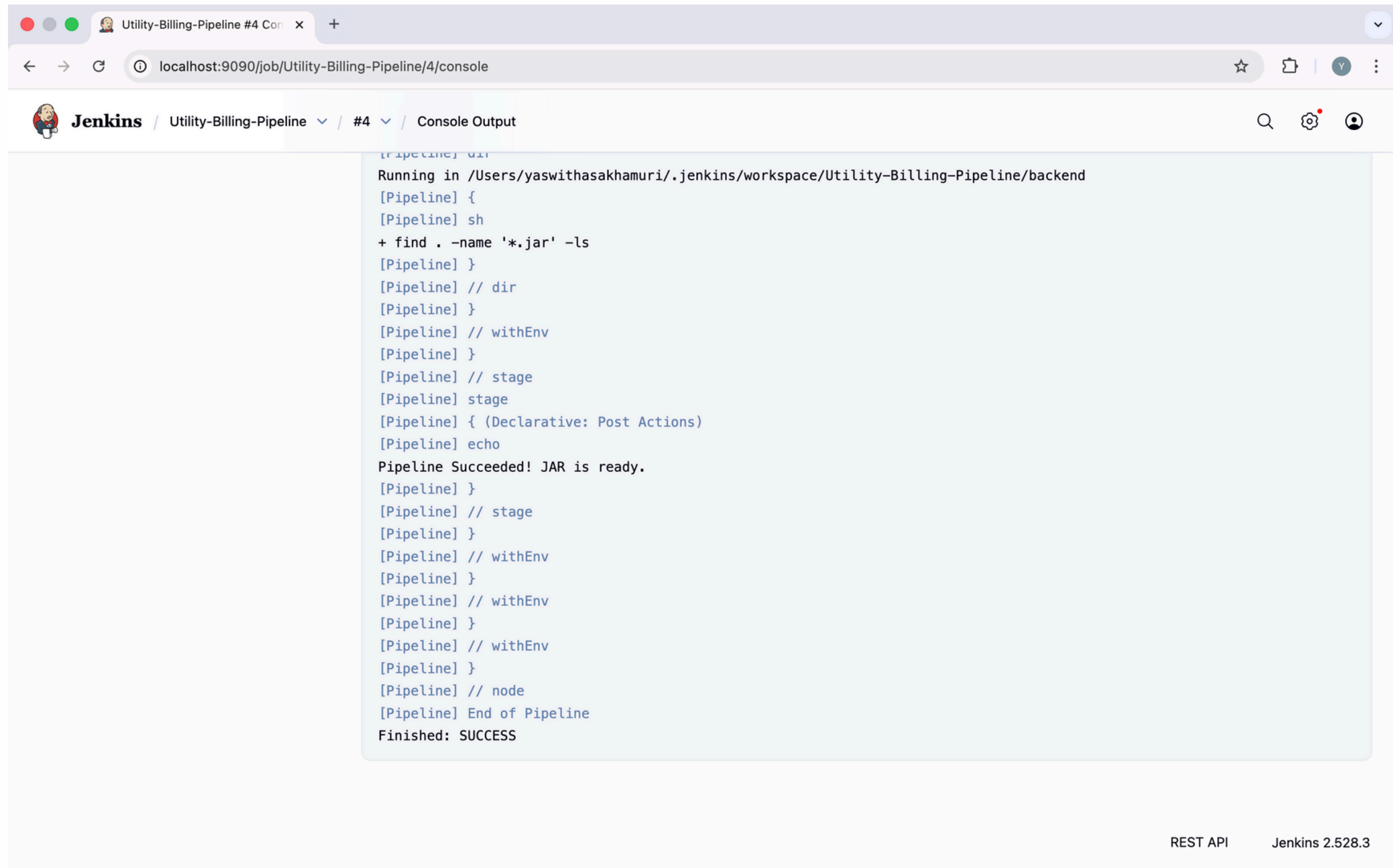
Engine running

RAM 3.54 GB CPU 2.38% Disk: 7.88 GB used (limit 223.63 GB)

Terminal

Update available

Jenkins Pipeline



The screenshot shows a web browser window with the address bar displaying `localhost:9090/job/Utility-Billing-Pipeline/4/console`. The Jenkins interface includes a breadcrumb trail: **Jenkins** / Utility-Billing-Pipeline / #4 / Console Output. The console output is as follows:

```
[Pipeline] dir
Running in /Users/yaswithasakhamuri/.jenkins/workspace/Utility-Billing-Pipeline/backend
[Pipeline] {
[Pipeline] sh
+ find . -name '*.jar' -ls
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline Succeeded! JAR is ready.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

At the bottom right of the page, the text "REST API" and "Jenkins 2.528.3" are visible.

Thank You